What's new

Modular Arithmetic Challenge

Terence Tao · 8 June 2026

https://terrytao.wordpress.com/2026/06/08/modular-arithmetic-challenge/

tao2typ · converted to Typst 2026-10-14 (UTC)

A couple months ago, Damek Davis and I launched the first mathematical challenge at the SAIR Foundation, aimed at "distilling" the ability to solve 22 million problems in universal algebra into a condensed form. Stage one of that challenge has now been completed, with several effective "cheat sheets" generated to guess the truth or falsity of these problems to reasonable accuracy; the leaderboard for that stage, with their winning cheatsheets can be found here. Stage two of that challenge, in which the competitors now have access to Python code as well as modest LLMs, and now need to generate Lean proofs or disproofs rather than just true-false answers, is currently underway.

With Alberto Alfarano, François Charton, Yongzheng Jia, Kristin Lauter, Cathy Li, and Emily Wenger, are launching a second challenge at SAIR, this time focused on seeing how efficiently neural networks can execute simple modular arithmetic operations. For this challenge we are focusing on the simple operation of modular multiplication: taking a prime modulus p (up to about a thousand digits long) and two integers a and b between 0 and $p-1$, and computing the product $a \cdot b \pmod p$. This is of course a solved problem using traditional computation, being a single line of code in any modern programming language. But it has been a fascinating toy problem in which to explore the basic capabilities of neural networks.

For instance, this problem has revealed the mysterious phenomenon of "grokking". When one tries to train a neural network on this problem for small sizes of inputs a, b, p, then initially one runs into the familiar problem of overfitting: the network learns to solve the problem for the training data too well, at the expense of performing well for held-out test data. However, if one continues training for sufficiently long periods of time, then the network can suddenly "grok" the problem and generalize surprisingly well to the test data. It appears that the neural network can suddenly "learn" powerful computational tricks, such as taking discrete logarithms, to find accurate and efficient ways to arrive at the correct answer.

This challenge is not about grokking, but instead about scaleability: we can create neural network models for modular multiplication that are extremely accurate for, say, 10-bit inputs, but they struggle at handling larger bit sizes. The competition is then simple: submit a neural network (with fixed weights) that can solve this task for larger input sizes with as high an accuracy as possible. Some pre-processing of the individual inputs a, b, p is permitted (e.g., to convert these numbers into decimal or some other convenient representation), but other than that the main computation has to be neural in nature; one cannot simply run some Python code, for instance, to compute the multiplication. We are imposing limits on the size and allocated run time on the neural network, but otherwise we are deliberately being flexible in the architecture requirements, in order to encourage creative experimentation; in particular, we permit networks whose weights were arrived at by other means than the usual machine learning training process.

This is a relatively simple challenge to state, but we genuinely do not know what to expect from the competitor entries – is there a clever way to encode modular arithmetic for even quite large numbers into a medium size neural network, or is it going to be an exceptionally difficult task? Hopefully we will find out in a few months! Discussion of the ongoing challenge will take place on this Zulip.

Comments (15)

Boris Alexeev · 8 June 2026

This challenge reminds me of https://codegolf.stackexchange.com/questions/183036/can-a-neural-network-recognize-primes

Anonymous · 9 June 2026

I do think discrete logarithm has an easy algorithm.

> **Terence Tao** · 9 June 2026
>
> Yes, we did discuss this as a possible "exploit" to the challenge; but for larger moduli (e.g., 1000 bits), we expect the computation of the discrete logarithm to be prohibitively expensive for the time budget allocated for competition submissions (which is to solve 1100 modular multiplication problems – each with a different modulus – in 5 minutes).
>
> > **Anonymous** · 9 June 2026
> >
> > here too smooth moduli failures may illuminate prime moduli failure

Jon Awbrey · 9 June 2026

There are alternative models of neural networks which do not depend on threshold neurons and endlessly fiddling with weights.

> **Terence Tao** · 9 June 2026
>
> We decided to be permissive as to what counts as a "neural network" to encourage creative solutions to the challenge. It is theoretically possible that one might be able to hand-code an efficient arithmetic circuit into such a network that basically performs a traditional computation, in which case we have to carefully adjudicate whether such a submission would be in both the letter and the spirit of the competition, but we decided not to prohibit such edge cases, at least initially, as such an encoding might be independently interesting.

Christopher · 9 June 2026

From the website "For a neural network, it is an open problem."
It seems like a stretch to consider this an open problem? The rules are subject to interpretation of course, but "On the Turing Completeness of Modern Neural Network Architectures" https://arxiv.org/abs/1901.03429 would seem to prove it is possible, even if they didn't apply it to this specific problem. So although it is an interesting "engineering" project I don't think I'd consider the problem open.
Or if that paper and previous papers doesn't close the open problem, it would be useful for the website to explain why!

> **Terence Tao** · 9 June 2026
>
> Fair enough; we should insert the word "efficiently" at the end of the basic question. We'll get that changed.
>
> > **Anonymous** · 14 June 2026
> >
> > Dr Tao, in your paper you used the concentration of measure result for the uniform random variable Y_H, whereas we need the concentration of measure for the random variables $Y_H | \ X_H = x$ for each x. The independence condition for primes p now seems to be nontrivial for each individual x. Can you explain this in slightly more detail?



Terence Tao · 16 June 2026

It is true that the random variables $Y_H \mid X_H = x$ are no longer uniformly distributed, and so concentration of measure theorems cannot directly be applied to that variable. However, because of (3.14), we can still transfer concentration results from the uniform variable to the non-uniform variable, at the cost of a significantly worse bound: see Lemma 3.3.



Anonymous · 9 June 2026

what would be better is discrete log modulus is smooth. We low there is an east algorithm. But may be nn does find something else which solves for smooth moduli in a different way in a probabilistic sense (say 2/3 success)which might illuminate the difficult prime moduli case through the 1/3 failures because 2/3 success is no fluke.



Anonymous · 16 June 2026

On the challenge website it says you get a, b and p as input and then need to output $a*b \mod p$ as a list of base-b digits. This seems very weird since b can be 100s of (base-10) digits long according to the website. What kind of insanely long alphabet am I supposed to use to represent these base b digits? Also it breaks the nice symmetry between a and b. What is the reason for this strange requirement?



Anonymous · 16 June 2026

Okay, reading further along the website I figured out that the b in base-b is supposed to be a different b, to be chosen by the contestant, than the b given as input. That makes things a bit easier. But my initial confusion was real. Perhaps it is possible to change this and use two different letters?
[We have updated the website to disambiguate these quantities. -T.]



Anonymous · 19 June 2026

Thanks! Also I didn't intend to write this anonymous, but I somehow misunderstood the comment system



Anonymous · 7 July 2026

For a beautiful communication of the grokking going on with Modular arithmetic, see the Welch Labs youtube video